中国科学技术大学软件学院

软件工程实验项目环节

开题报告

项 目 名 称：基于多智能体协同的科研文献智能综述生成系统

成 员 名 单：待填写（项目组 4 人）

导　　师：待填写

工 程 领 域：软件工程

研 究 方 向：智能软件工程 / 大语言模型应用

开 题 时 间：2026 年 10 月

中国科学技术大学软件学院

填表日期：2026 年 10 月 27 日

一、简况

表 1 项目名称

项目名称	内容
中文	基于多智能体协同的科研文献智能综述生成系统
英文	A Multi-Agent Collaborative System for Intelligent Literature Review Generation

表 2 项目组成员名单

姓名	学号	项目中的分工	签章
待填写（负责人）	待填写	总体统筹、CrewAI 编排、任务分解与结果融合	
待填写	待填写	文献检索、数据治理、RAG 与向量索引	
待填写	待填写	后端 API、任务调度、前端集成与部署	
待填写	待填写	评测设计、实验执行、技术报告与演示	

中文摘要

针对科研文献数量快速增长、综述撰写依赖人工且引用难以核验的问题，本项目拟设计并实现一个基于 CrewAI 的多智能体协同文献综述生成系统。系统以自然语言研究主题为输入，由协调、检索、分析、对比、撰写和引用校验智能体共同完成检索规划、论文筛选、方法抽取、演进脉络分析与结构化写作。系统采用面向科研文本的混合 RAG 策略：对 arXiv、PubMed 等来源的文献进行元数据规范化、章节感知切分，结合 BM25 与稠密向量检索、重排序和证据定位，将报告中的关键声明绑定到可回溯的原文证据。项目还将设计基于有向无环任务图的任务分解、结构化产物交换和冲突消解机制，避免多智能体自由对话导致的状态漂移与错误累积。在工程实现上，系统将提供 Web 交互原型，支持检索条件配置、生成进度监控、证据查看、报告编辑与多格式导出。评测将在检索增强生成、大语言模型多智能体系统、软件漏洞检测三个计算机子方向开展，与单智能体、无 RAG 等基线比较，重点考察引用准确性、证据可追溯性、内容覆盖度、结构完整性和专家满意度。预期形成可运行系统、评测工具、技术报告和可复现实验材料，为可信科研文献辅助工具提供可借鉴的工程方案。

English Abstract

To address the rapid growth of scientific literature and the difficulty of producing reviews with verifiable citations, this project designs and implements a CrewAI-based multi-agent system for intelligent literature review generation. Given a natural-language research topic, a coordinator, retrieval, analysis, comparison, writing, and citation-verification agents collaborate on search planning, paper screening, method extraction, research-evolution analysis, and structured writing. The system applies a scientific-text-oriented hybrid RAG strategy: it normalizes metadata from sources such as arXiv and PubMed, performs section-aware chunking, combines BM25 and dense retrieval, reranks candidate evidence, and binds key claims in the report to stable source snippets and locators. A directed acyclic task graph, typed artifact exchange, and conflict-resolution mechanism are designed to avoid state drift and error accumulation in free-form multi-agent conversations. The engineering work includes a web prototype that supports constraint configuration, progress monitoring, evidence inspection, report editing, and multi-format export. Evaluation will cover three computer-science subfields: retrieval-augmented generation, LLM-based multi-agent systems, and software vulnerability detection. It will compare the proposed system with single-agent and no-RAG baselines using citation precision, evidence traceability, content coverage, structural completeness, expert satisfaction, latency, and cost. The expected outcomes are a deployable prototype, evaluation tooling, a technical report, and reproducible experimental materials that provide an engineering reference for trustworthy research assistants.

主题词：多智能体系统；检索增强生成；智能文献综述

Keywords：Multi-agent Systems / Retrieval-Augmented Generation / Intelligent Literature Review

> 说明：项目名称、摘要与主题词已填写；成员姓名、学号、导师等信息未知，暂以“待填写”占位，提交前请按实际名单替换。

二、选题依据

2.1 研究背景

研究背景概览

- 文献供给侧：论文数量多、跨库分散，全文与元数据质量不一致。
- 研究需求侧：需要快速建立方法分类、代表工作、实验条件与演进脉络。
- 技术可行性：RAG 提供外部证据，多智能体支持角色化任务分解与协作。
- 工程目标：把检索、分析、比较、写作和引用校验组织为可观测工作流。

科研文献的产出速度已经超过个人持续跟踪的能力。研究者在开展一个新课题时，通常需要经历主题拆解、数据库检索、论文筛选、方法抽取、结果对比和综述写作等环节；这些环节既依赖领域知识，也包含大量可重复但耗时的手工操作。传统综述往往由专家在数月内完成，普通课程项目或初入领域的研究者则容易受到检索遗漏、分类标准不一致和引用过期等问题的影响。随着预印本数量增加，单一数据库和一次性检索更难覆盖一个方向的主要工作，综述过程的透明性和可复现性也因此变得重要。

大语言模型（Large Language Model, LLM）为文献阅读和写作提供了新的自动化手段，但直接使用生成式模型存在两类核心风险：一是模型可能生成没有来源支撑的结论；二是长上下文中的关键信息容易被忽略或混淆[11]。检索增强生成（Retrieval-Augmented Generation, RAG）通过先检索外部知识、再将证据放入生成上下文，为提高事实性和可溯源性提供了可行路径[1][2]。与此同时，多智能体系统把复杂任务拆分为多个具有明确角色的子任务，使检索、分析、比较和写作可以由不同的智能体承担，并通过协作完成单智能体难以稳定处理的流程[4][5]。因此，将 CrewAI 的多智能体编排能力与面向科研文本的 RAG 结合起来，构建一个可追溯、可评测的文献综述生成系统，具有明确的研究和工程价值。

2.2 研究问题与意义

本项目关注的核心问题不是“让模型一次写出一篇文章”，而是“如何让一个多智能体系统在有限时间与预算内，稳定地产出覆盖关键工作、结构清晰且引用可核验的文献综述”。围绕这一问题，需要同时回答任务如何分解、证据如何检索、智能体结果如何融合、错误如何发现以及质量如何评测等子问题。现有系统往往强调生成结果的表面流畅性，却缺少对证据链、冲突观点和失败恢复的工程化处理；本项目试图把综述生成从一次性提示词调用转化为可观测、可回滚的工作流。

从理论角度看，本项目有助于探索 LLM 多智能体在长流程知识工作中的任务分解与结果融合方法，特别是结构化产物交换、证据约束生成和冲突消解对系统可靠性的影响。从应用角度看，系统可以降低研究者完成初步文献调研的时间成本，帮助课程学习者快速建立领域地图，并为课题组内部的知识管理提供可复用工具。相比只追求生成速度的“自动写作”工具，强调引用准确性和可复现性的系统更适合科研场景，也更容易被导师、审稿人和领域专家接受。

2.3 国内外研究现状

2.3.1 检索增强生成

RAG 的基本思想是在生成前检索与问题相关的外部文档，并把检索结果作为条件上下文。Lewis 等提出的 RAG 将参数化语言模型与非参数化文档索引结合，在开放域问答等知识密集型任务上验证了外部检索的价值[1]。此后，研究重点逐渐从“是否检索”转向“何时检索、检索什么、如何验证”。Gao 等的综述从检索、生成和增强三个层面梳理了 RAG 的技术路线[2]；Huang 和 Huang 进一步从文本生成角度总结了稠密检索、重排序、上下文压缩和评估方法[3]。Self-RAG 通过让模型判断是否需要检索并对生成内容进行自我反思，说明了生成过程中的动态控制价值[9]。

在科学研究场景中，RAG 面临的困难比通用问答更突出。论文包含摘要、方法、实验设置、结果表格和限制讨论等多个层次，简单按固定长度切分会破坏论证结构；不同论文的实验条件和评价指标也不一致，不能仅凭相似度把结论拼接在一起。PaperQA 等工作尝试将检索、引用和科学问答结合，说明面向论文的检索需要同时考虑文本片段、来源元数据和可验证性[14]。本项目将采用“元数据过滤 + 混合检索 + 重排序 + 证据定位”的组合策略，并把引用校验从写作后的附加步骤提升为生成流程中的约束条件。

2.3.2 大语言模型多智能体协作

LLM 多智能体系统通常通过角色设定、消息传递和任务分工，把复杂目标拆解为多个可管理的子任务。AutoGen 提供了多智能体对话与工具调用的通用框架[5]；MetaGPT 将标准作业程序引入多智能体协作，强调结构化中间产物对复杂任务的作用[6]；ChatDev 则通过角色分工和通信机制完成软件开发生命周期中的多项任务[7]。这些工作说明，多智能体的优势不仅来自“更多模型”，更来自角色边界、任务契约和可检查的中间结果。

不过，多智能体协作也会带来新的工程问题。自由对话容易导致上下文膨胀、职责漂移和错误级联；不同智能体可能重复检索或给出相互冲突的结论；在长流程中，如果没有统一的产物格式和状态管理，系统很难定位失败发生在哪一步[4]。CrewAI 以角色、目标和任务为核心抽象，支持顺序或层级式协作，适合快速搭建原型；但要把 CrewAI 用于科研写作，仍需要额外的任务图、证据

库、结构化 schema 和校验器。本项目因此不把智能体之间的自然语言聊天作为唯一协作方式，而是采用“角色 + 任务 + 结构化产物”的混合编排。

2.3.3 自动文献综述与科研助手

自动综述是 LLM 科研辅助中的代表性任务。LitLLM 将文献检索、摘要和综述草稿生成组织为工具链，强调以来源文献为基础的写作流程[15]；AutoSurvey 通过检索、组织大纲、分段生成和迭代修订等步骤，探索了大规模综述的自动生成方法[16]；Agent Laboratory 进一步把文献综述、实验和报告写作纳入研究助理式工作流[17]。这些系统证明了端到端自动综述的可行性，也暴露出一些共性不足：引用往往停留在文献级别，缺少“声明—证据—定位”的细粒度绑定；生成过程的可观测性有限；对不同子领域的评测还不够系统。

2.3.4 国内相关研究概况

国内高校和研究机构在 LLM 智能体、代码智能和自动综述方向已有较多工作。ChatDev、MetaGPT 和 AutoSurvey 等代表性研究中都有国内团队参与，研究重点从角色协作扩展到结构化任务规划、软件工程流程和长文本组织[6][7][16]。产业界也出现了面向检索、知识库问答和报告生成的工具，但多数产品更强调交互体验和通用写作，对科研文本的章节结构、实验条件、引用定位和冲突观点处理仍不充分。总体来看，国内研究与国外研究几乎同步，真正缺少的是把多智能体、RAG 和可验证评测整合起来的开源工程原型。

表 3 与项目直接相关的代表性研究及启示

研究方向	代表工作	主要贡献	对本项目的启示
检索增强生成	RAG、Self-RAG、PaperQA、RAGAS/ARES	引入外部证据、动态检索和自动评估；证明证据约束能改善事实性	检索必须与引用验证联动；评估应覆盖检索、生成和引用三个层面
LLM 多智能体	AutoGen、MetaGPT、ChatDev、CrewAI	角色分工、任务协作、结构化中间产物和工具调用	用任务契约和产物 schema 约束协作，避免自由对话导致状态漂移
自动综述	LitLLM、AutoSurvey、Agent Laboratory	覆盖检索、组织、生成和迭代修订的端到端流程	需要细粒度 claim-evidence 绑定、冲突消解和跨领域评测
科研问答与知识管理	PaperQA 等	面向论文语料的检索、问答和引用展示	论文切分应保留章节、表格、页码等定位信息

2.4 现有不足与发展趋势

现有自动综述系统的主要不足可以归纳为四点。第一，引用粒度偏粗。系统经常在段落末尾列出若干参考文献，但读者无法判断某一条具体结论由哪一篇论文、哪一段原文支持。第二，任务融

合机制偏弱。多智能体系统更多依赖对话历史传递信息，缺少统一的证据对象、版本化产物和冲突处理规则。第三，评测范围有限。很多工作只在单一领域或少量主题上比较生成文本的流畅度，缺少对引用可靠性、覆盖度和跨领域泛化的联合评估。第四，工程闭环不完整。部分研究停留在离线脚本或演示页面，尚未覆盖任务恢复、权限控制、导出、成本监控和用户编辑等实际需求。

未来趋势是从“文本生成”走向“证据驱动的知识工作流”：检索、分析、比较、写作和验证将成为可观测的流水线；智能体之间将通过结构化对象交换信息；人类可以在关键节点进行审阅和纠错；评测将从单一 BLEU/ROUGE 等文本相似度指标转向引用准确率、事实一致性、覆盖度、专家评分和成本效率的组合指标[12][13][18]。本项目正是沿着这一趋势，设计面向科研文献的多智能体协作与混合 RAG 方案。

2.5 调研方法与文献筛选

本阶段采用范围式文献调研与滚雪球检索相结合的方法。检索范围以 2020—2026 年的 arXiv、Crossref、ACL Anthology、NeurIPS/ICLR/ACL 等会议论文和 CrewAI 官方文档为主，关键词包括 retrieval-augmented generation、multi-agent LLM、automated literature review、scientific document understanding、citation verification、agent evaluation 等。筛选时优先保留与 RAG、多智能体协作、自动综述和科研问答直接相关的工作；对于基础方法，补充检索其原始论文和权威综述；对于新近预印本，重点关注是否提供可复现实验、开源代码或被后续工作引用。

调研过程按“问题—方法—证据—局限”四列整理文献：先记录研究问题和任务设置，再提取检索/生成/编排方法，随后核对实验数据、指标和结论，最后标注局限及与本项目的差异。该过程不是一次性穷举，而是随系统原型和评测结果迭代补充。当前调研的结论是：RAG 和多智能体分别有较成熟的技术基础，但两者的工程化整合、细粒度引用验证和跨子领域评测仍有明显空白，适合作为软件工程实验项目的切入点。

2.6 选题的先进性与实用性

本选题的先进性体现在三个层面。其一，把多智能体协作从“角色对话”推进到“任务图 + 结构化产物 + 共享证据库”的工程模式，使每个阶段都有明确输入、输出和验收条件。其二，将混合检索、重排序、证据定位和引用校验组合为面向科研文本的 RAG 流水线，要求关键声明在生成时绑定 evidenceId，而不是在写作后机械补参考文献。其三，设计跨三个计算机子方向的评测方案，同时考察质量、引用、时延和成本，避免只用单一文本指标判断系统优劣。

实用性体现在系统面向真实的文献调研流程：用户可以输入自然语言主题，选择时间范围和数据源，查看生成进度，打开任意引用查看原文片段，编辑报告并导出 DOCX、PDF 或 Markdown。系统不替代研究者做最终判断，而是把重复的检索、整理和初稿搭建工作自动化，把人的精力集中到

问题定义、证据审查和观点提炼上。对于课程项目而言，系统也提供了从需求分析、UML 建模、后端服务、前端交互到质量评测的完整软件工程实践。

2.7 技术难点与工作量

本项目的技术难点主要来自五个方面。第一，学术数据接入存在格式差异和访问限制，需要处理 API 限流、分页、元数据缺失以及开放全文与摘要的边界。第二，论文解析需要保留章节、页码、表格和公式等结构信息，否则证据定位会失真。第三，混合检索和重排序需要在召回率、上下文长度和成本之间权衡，长上下文中的“中间信息丢失”会直接影响生成质量[11]。第四，多智能体错误会沿任务链传播，必须通过 schema 校验、置信度阈值、重试和人工复核控制风险。第五，综述质量具有主观性，需要设计可操作的评价量表、盲评流程和评分者一致性检验。

工作量估计包括：文献与需求调研约 3 周；系统架构、数据模型和 UML 设计约 2 周；数据接入、解析和 RAG 流水线约 6 周；CrewAI 编排、任务图、融合与校验约 5 周；Web 前后端与部署约 4 周；三领域评测、消融实验和报告撰写约 4 周。整体按 24 周安排，其中预留约 2 周用于联调、缺陷修复和答辩准备。项目工作量与一名本科生/研究生课程项目的周期基本匹配，但要求团队在数据治理和评测设计上投入足够精力。

参考文献

[1] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems 33. 2020: 9459-9474. arXiv:2005.11401.

[2] GAO Y, XIONG Y, GAO X, et al. Retrieval-Augmented Generation for Large Language Models: A Survey[EB/OL]. arXiv:2312.10997, 2023.

[3] HUANG Y, HUANG J X. A Survey on Retrieval-Augmented Text Generation for Large Language Models[J]. ACM Computing Surveys, 2026, 58(12): 1-38. DOI: 10.1145/3805774.

[4] WANG L, MA C, FENG X, et al. A Survey on Large Language Model Based Autonomous Agents[J]. Frontiers of Computer Science, 2024, 18(6). DOI: 10.1007/s11704-024-40231-1.

[5] WU Q, BANSAL G, ZHANG J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation[EB/OL]. arXiv:2308.08155, 2023.

[6] HONG S, ZHUGE M, CHEN J, et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework[EB/OL]. arXiv:2308.00352, 2023.

[7] QIAN C, LIU W, LIU H, et al. ChatDev: Communicative Agents for Software Development[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. 2024: 15174-15186. DOI: 10.18653/v1/2024.acl-long.810.

[8] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[EB/OL]. arXiv:2210.03629, 2022.

[9] ASAI A, WU Z, WANG Y, et al. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection[EB/OL]. arXiv:2310.11511, 2023.

[10] ROBERTSON S, ZARAGOZA H. The Probabilistic Relevance Framework: BM25 and Beyond[J]. Foundations and Trends in Information Retrieval, 2009, 4(1-2): 1-174. DOI: 10.1561/1500000019.

[11] LIU N F, LIN K, HEWITT J, et al. Lost in the Middle: How Language Models Use Long Contexts[J]. Transactions of the Association for Computational Linguistics, 2024, 12: 157-173. DOI: 10.1162/tacl_a_00638.

[12] ES S, JAMES J, ESPINOSA ANKE L, et al. RAGAS: Automated Evaluation of Retrieval Augmented Generation[C]//Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations. 2024: 150-158. DOI: 10.18653/v1/2024.eacl-demo.16.

[13] SAAD-FALCON J, KHATTAB O, POTTS C, et al. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems[C]//Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2024: 338-354. DOI: 10.18653/v1/2024.naacl-long.20.

[14] LÁLA J, O'DONOGHUE O, SHTEDRITSKI A, et al. PaperQA: Retrieval-Augmented Generative Agent for Scientific Research[EB/OL]. arXiv:2312.07559, 2023.

[15] AGARWAL S, SAHU G, PURI A, et al. LitLLM: A Toolkit for Scientific Literature Review[EB/OL]. arXiv:2402.01788, 2024.

[16] WANG Y, GUO Q, YAO W, et al. AutoSurvey: Large Language Models Can Automatically Write

Surveys[C]//Advances in Neural Information Processing Systems 37. 2024: 115119-115145. DOI: 10.52202/079017-3655.

[17] SCHMIDGALL S, SU Y, WANG Z, et al. Agent Laboratory: Using LLM Agents as Research Assistants[C]//Findings of the Association for Computational Linguistics: EMNLP 2025. 2025: 5977-6043. DOI: 10.18653/v1/2025.findings-emnlp.320.

[18] FABBRI A R, KRYŚCIŃSKI W, MCCANN B, et al. SummEval: Re-evaluating Summarization Evaluation[J]. Transactions of the Association for Computational Linguistics, 2021, 9: 391-409. DOI: 10.1162/tacl_a_00373.

[19] CREWAI INC. CrewAI Documentation[EB/OL]. https://docs.crewai.com/, 2026-10-27.

[20] SHINN N, CASSANO F, GOPINATH A, et al. Reflexion: Language Agents with Verbal Reinforcement Learning[EB/OL]. arXiv:2303.11366, 2023.

[21] LIU X, YU H, ZHANG H, et al. AgentBench: Evaluating LLMs as Agents[EB/OL]. arXiv:2308.03688, 2023.

[22] KRIPPENDORFF K. Content Analysis: An Introduction to Its Methodology[M]. 4th ed. Los Angeles: SAGE, 2018.

三、课题内容及具体方案

3.1 课题内容

3.1.1 研究目标

本项目的总体目标是：面向科研文献综述这一长流程知识任务，设计并实现一个基于 CrewAI 的多智能体协同系统，使系统能够根据用户输入的自然语言主题，自动完成文献检索、内容分析、方法对比、结构化写作和引用校验，并通过 Web 应用提供可交互、可追溯、可导出的综述生成服务。系统不仅要在演示场景中生成流畅文本，还要在证据可核验性、任务可恢复性和跨子领域评测上达到课程项目可验证的质量水平。

具体目标包括：（1）建立至少包含检索、分析、对比、撰写和引用校验的多智能体角色体系，并明确每个角色的输入、输出和失败处理方式；（2）实现一种面向科研文本的混合 RAG 策略，覆盖元数据规范化、章节感知切分、BM25 与稠密检索、重排序、证据定位和引用绑定；（3）设计任务分解与结果融合机制，使多智能体协作从自由对话转变为可观测的任务图执行；（4）完成端到端 Web 原型，支持主题配置、过程监控、证据查看、报告编辑和多格式导出；（5）在三个计算机子方向上完成与单智能体、无 RAG 等基线的对比实验，形成技术报告和可复现材料。

3.1.2 研究问题

围绕总体目标，本项目将以下研究问题作为设计与评测的主线。

RQ1 如何把文献综述任务分解为具有明确输入、输出和验收条件的智能体任务，并在任务失败时进行恢复？

RQ2 面向科研论文的混合 RAG 策略应如何组合元数据过滤、稀疏检索、稠密检索和重排序，才能提高引用准确性和证据覆盖率？

RQ3 如何融合检索、分析、对比和写作智能体的结构化产物，并在冲突时保留不同观点而不是强行给出单一结论？

RQ4 在检索增强生成、LLM 多智能体系统、软件漏洞检测三个子方向上，系统的质量、时延和成本如何权衡？

RQ5 如何把上述方法落地为可操作、可观测、可回滚的 Web 应用，并满足课程项目的工程规范？

3.1.3 研究范围与边界

系统首期面向英文科研论文的检索与中英文综述生成，数据源优先接入 arXiv 和 PubMed，辅以 Crossref 等元数据服务。单次任务的候选论文规模控制在 40—80 篇，经过筛选后纳入 20—40

篇；报告长度可配置为 3000—8000 字。系统覆盖研究背景、主流方法分类、技术演进、代表性工作比较、开放问题和参考文献等章节，但不替代领域专家对研究价值的判断，也不承诺自动发现所有相关文献。对于无法获得开放全文的论文，系统只使用摘要和公开元数据，并在报告中明确标注证据范围。

系统采用人机协同原则：用户可以调整检索条件、删除不相关论文、编辑报告并查看引用证据；系统应保留每次生成使用的模型、提示词、检索参数和语料快照，以支持复现。由于大语言模型可能产生错误，最终报告在提交或发表前必须经过人工审查。项目不存储用户未授权的私人文献，不绕过数据库访问控制，不把受版权保护的全文重新分发。

3.1.4 主要研究内容

主要研究内容包括五个部分：其一，多源学术数据接入与论文规范化；其二，面向科研文本的混合检索与证据库；其三，基于 CrewAI 的角色智能体与任务图编排；其四，结构化结果融合、声明—证据绑定和引用校验；其五，Web 原型与三领域评测。五个部分相互依赖：数据质量决定检索上限，检索质量决定分析依据，分析产物决定写作质量，引用校验决定可信度，评测结果反过来驱动任务分解和提示策略的迭代。

3.2 系统需求分析

3.2.1 用户与使用场景

系统的主要用户是需要在较短时间内建立领域认识的研究生、本科生和科研人员；次要用户包括课程教师、课题组负责人以及负责部署和维护系统的管理员。典型场景包括：开题前快速梳理一个方向的主要方法；撰写论文 related work 时查找可引用的代表工作；比较不同方法的数据集、指标和实验条件；在组会上生成一份可讨论的综述初稿。系统输出是“可编辑的研究材料”，不是最终学术结论。

表 4　利益相关者与需求关注点

利益相关者	主要目标	关键关注点	系统响应
研究者/学生	快速形成领域地图，获取可引用材料	覆盖度、引用准确、可编辑、导出方便	主题配置、证据面板、报告编辑、多格式导出
教师/导师	检查学生调研过程与证据质量	过程透明、引用可追溯、可复现	任务日志、版本历史、评测报告
系统管理员	部署、配置和监控系统	稳定性、权限、成本和数据源配置	模型/数据源配置、任务监控、失败重试

3.2.2 功能需求

表 5 功能需求及验收要点

编号	功能需求	优先级	验收要点
FR-01	接收自然语言研究主题，并解析时间范围、数据源、语言和篇幅等约束	P0	缺失字段有默认值;提交后生成任务编号
FR-02	从 arXiv、PubMed 等来源检索论文，并支持分页、限流和失败重试	P0	每次检索记录查询语句、时间和返回数量
FR-03	对论文进行元数据规范化、DOI/arXiv ID 去重和相关性筛选	P0	重复记录合并；筛选理由可查看
FR-04	对论文进行章节感知切分，并建立 chunk、页码/段落定位和嵌入索引	P0	每个证据片段可回到原文位置
FR-05	提供 BM25 与稠密向量的混合检索及重排序	P0	可配置召回数量、重排数量和过滤条件
FR-06	提取每篇论文的方法、数据集、指标、创新点和局限	P0	输出符合 AnalysisArtifact schema
FR-07	按方法、数据集、指标和时间等维度进行对比，识别演进与冲突	P0	输出 ComparisonArtifact 和证据引用
FR-08	生成包含背景、方法分类、演进路线、开放问题等章节的综述	P0	结构完整；关键声明有 evidenceId
FR-09	对声明—证据支持关系进行校验，并提供定向改写或人工复核	P0	校验结果可解释;失败项不被静默忽略
FR-10	查看报告中的引用和原文证据，支持按论文/章节筛选	P0	点击引用可打开证据面板
FR-11	支持报告编辑、版本保存和 Markdown/DOCX/PDF 导出	P1	导出内容与当前版本一致
FR-12	展示任务进度、失败原因、重试记录和成本统计	P1	任务可取消；失败可从检查点恢复

3.2.3 非功能需求

表 6 非功能需求

编号	类别	需求描述	目标/约束
NFR-01	性能	长任务异步执行，进度可增量推送	首次状态回显 < 5 s；单次 40—60 篇任务在正常 API 条件下 < 10 min
NFR-02	可靠性	任务可重试、可恢复，避免重复写入和状态丢失	关键任务幂等；失败任务保留最后检查点
NFR-03	可追溯性	引用必须指向稳定证据对象和原文定位	关键声明 evidence 绑定率 100%
NFR-04	可复现性	记录模型、提示词、检索参数和语料快照	同一配置可重放主要流程
NFR-05	安全性	密钥不进入前端和日志；用户数据按项目隔离	RBAC + 环境变量/密钥管理；日志脱敏
NFR-06	可维护性	模块边界清晰，智能体输出使用 schema 校验	核心模块单元测试覆盖率目标 ≥ 70%
NFR-07	易用性	主要流程不超过三个页面层级；错误提示可行动	桌面端优先，兼容平板宽度
NFR-08	成本	限制单次 token 预算，复用缓存和中间结果	超预算任务暂停并提示用户确认

3.2.4 用例模型

图 1 给出系统用例图。研究者是主要参与者，系统管理员是次要参与者，学术数据库作为外部系统参与检索用例。生成综述用例通过 <<include>> 关系复用“检索并规范化文献”和“校验声明与引用”两个必需用例，体现了主流程对检索和验证的依赖。

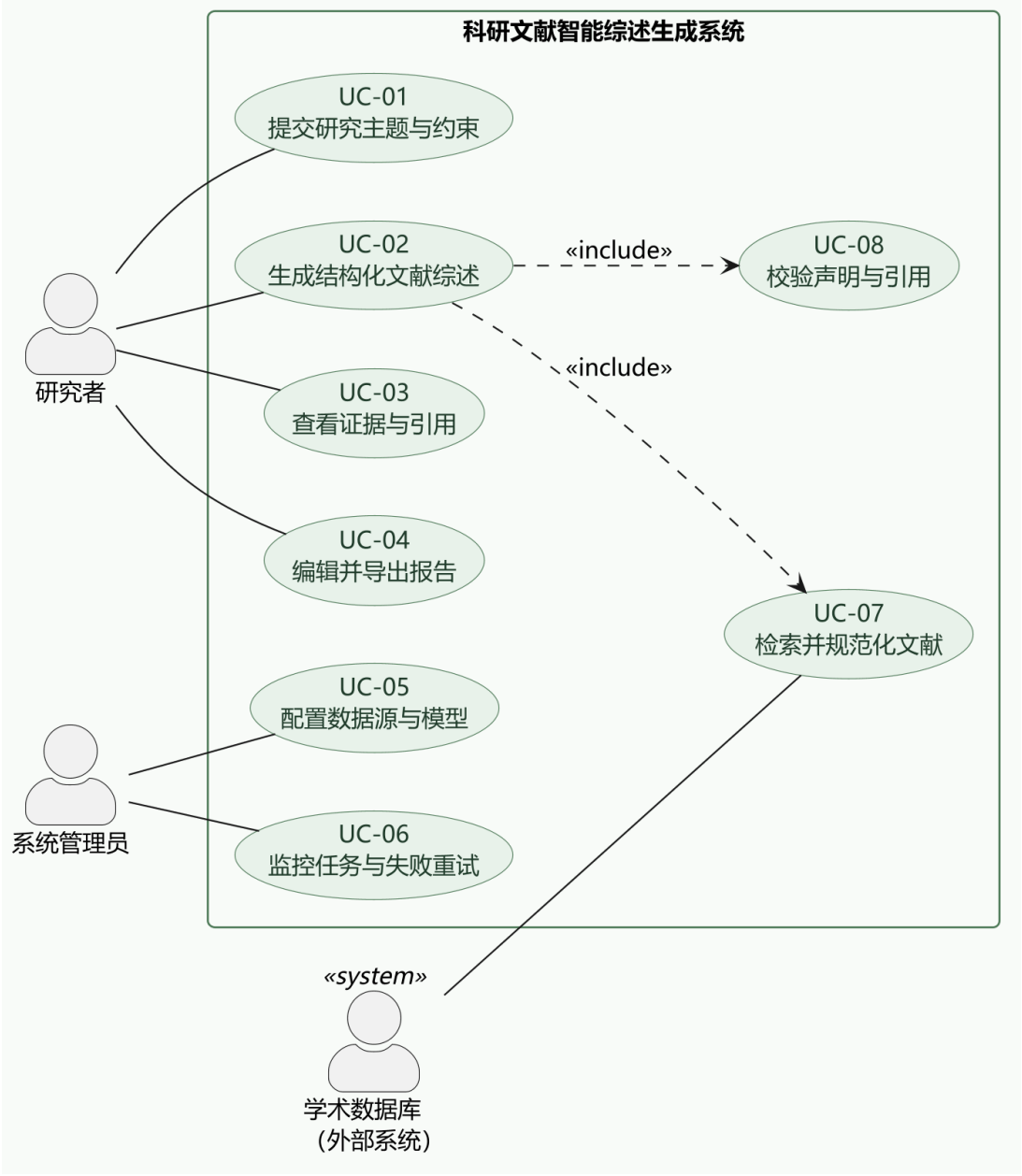


图 1 科研文献智能综述系统用例图

UML 建模约定：用例图中参与者位于系统边界外；<<include>> 表示基用例必然执行的复用行为；顺序图中实线箭头表示同步消息、虚线箭头表示返回消息，loop 和 alt 分别表示循环与条件分支；类图标注关联多重性和组合关系；活动图使用泳道、判定节点和循环回边。图中的类名与代码中的领域对象保持一致，便于从设计追踪到实现。

表 7 核心用例 UC-02 规约

项目	UC-02 生成结构化文献综述
主要参与者	研究者
前置条件	用户已登录；系统已配置至少一个学术数据源和可用 LLM；主题非空
触发条件	用户提交主题、时间范围、数据源、篇幅和语言等约束
主成功场景	1）系统解析约束并生成检索计划；2）检索智能体召回论文；3）系统去重和筛选；4）分析智能体抽取方法/数据集/指标；5）对比智能体归纳相似性、差异和演进；6）撰写智能体生成带 evidenceId 的大纲与初稿；7）校验智能体检查声明—证据关系；8）系统保存报告和引用图并向用户展示
扩展场景	检索失败：按退避策略重试并记录原因；证据不足：扩大检索或标记人工复核；校验失败：定向改写，超过重试上限则停止并展示问题清单
后置条件	生成一个可查看、可编辑、可导出的综述版本；引用均指向证据库中的记录

3.3 系统概要设计

3.3.1 设计原则

系统设计遵循五项原则。第一，证据优先：综述中的关键声明必须关联证据对象，不能只依赖模型记忆。第二，结构化协作：智能体之间传递带类型的产物，而不是无限增长的聊天记录。第三，确定性与灵活性结合：数据接入、去重、评分和校验由确定性代码完成，开放式归纳和写作交给 LLM。第四，人类可介入：检索条件、论文筛选、证据查看和报告编辑均保留人工操作入口。第五，可复现与可扩展：模型、提示词、检索参数和语料范围写入任务记录，数据源和模型通过适配器替换。

3.3.2 总体架构

系统采用分层架构，如图 2 所示。表现层提供主题输入、进度、报告和证据面板；应用服务层负责 REST API、任务调度和 CrewAI 工作流编排；智能体层包含协调、检索、分析、对比、撰写和引用校验六类角色；RAG 与数据层负责混合检索、重排序、证据库、向量索引、关系数据库和对象存储；外部服务层包括 arXiv、PubMed 和可替换的 LLM 服务。Web 前端不直接访问模型密钥，所有外部调用由后端 LLM 网关统一管理。

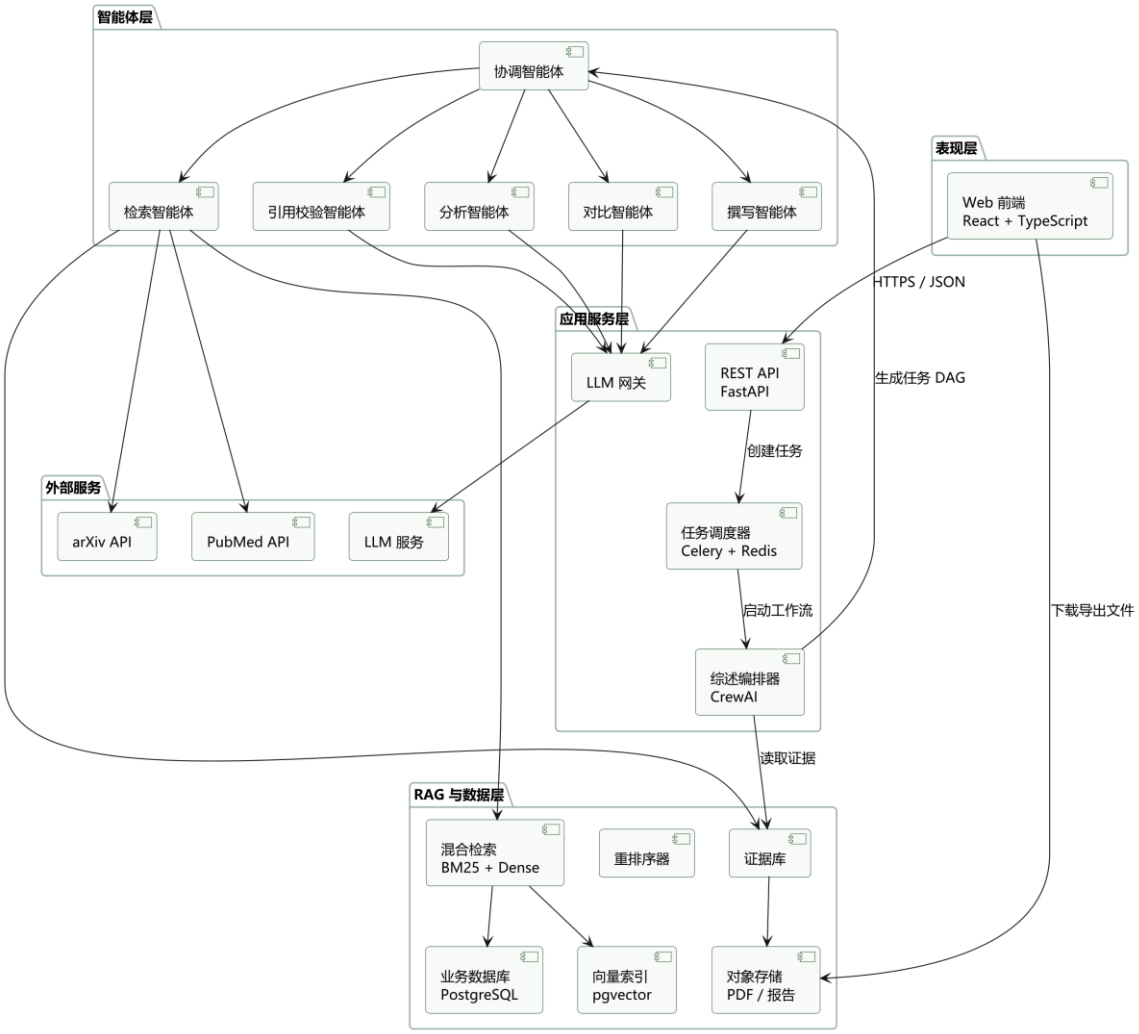


图 2 系统组件图

3.3.3 多智能体角色与 CrewAI 编排

系统使用 CrewAI 的 Agent、Task 和 Crew 抽象表达角色和目标，但不把整个系统简化为一次 crew.kickoff()。数据接入、任务状态、重试、预算和持久化由应用层控制；CrewAI 负责在受控的上下文中执行智能体任务并返回结构化结果。每个 Agent 配置角色、目标、工具和输出 schema；每个 Task 配置描述、期望输出、上下文依赖和最大重试次数；协调智能体根据主题生成任务图，编排器按依赖关系调度。对于可并行的逐论文分析任务，系统使用批次并行；对于需要全局视角的对比和写作任务，系统采用串行汇总。

表 8 智能体职责与结构化产物

智能体	主要职责	输入	输出（schema）	失败处理
协调智能体	解析主题、生成检索计划、分配任务与预算、汇总状态	用户约束、项目状态	PlanArtifact	回退到规则模板；请求用户补充条件
检索智能体	查询扩展、数据源调用、去重、筛选和证据收集	PlanArtifact	RetrievalArtifact	换查询/数据源重试；返回覆盖率警告
分析智能体	逐篇提取方法、数据集、指标、创新点和局限	Paper + chunks	AnalysisArtifact[]	schema 校验失败则重试或标记缺失
对比智能体	聚类方法、比较实验条件、识别演进和矛盾	AnalysisArtifact[]	ComparisonArtifact	增加证据或降级为描述性对比
撰写智能体	生成大纲、章节草稿、声明和引用占位	ComparisonArtifact + Evidence	DraftArtifact	按不完整章节分段重写
引用校验智能体	检查 claim-evidence 支持度、定位和引用格式	DraftArtifact + Evidence	VerificationArtifact	返回 unsupportedClaims，触发定向改写

3.3.4 任务分解机制

任务分解采用“阶段—任务—产物”三层结构。阶段对应检索、分析、对比、写作和校验；任务是最小调度单元，具有唯一编号、依赖列表、输入引用、约束、输出 schema、优先级、超时和重试策略；产物是不可变的版本化对象，写入共享黑板（blackboard）后供下游引用。任务图是有向无环图（DAG），允许分析任务并行、写作任务按章节并行，但所有合并节点必须等待依赖完成。

代码 1 TaskSpec 的结构示意

```
{
  "task_id": "T-analyze-001",
  "type": "PAPER_ANALYSIS",
  "input_refs": ["paper:2402.01788", "evidence:chunk-017"],
  "constraints": {"language": "zh", "max_tokens": 1800},
  "required_output": "AnalysisArtifact",
  "dependencies": ["T-retrieve-003"],
  "max_retries": 2,
  "priority": 5
}
```

任务状态机为 CREATED → PLANNING → RETRIEVING → ANALYZING → COMPARING → DRAFTING → VERIFYING → COMPLETED；任一阶段可进入 FAILED 或 CANCELLED，重试时从最近检查点恢复而不是重新执行全部流程。协调器在每阶段检查 token 预算、论文覆盖率和证据数量；若覆盖率不足，则扩展检索而不是让写作智能体凭空补全。共享黑板只接受通过 schema 校验的产物，避免无效文本污染下游上下文。

3.3.5 结果融合与冲突消解

结果融合分为五步：规范化、去重、聚类、冲突检测和证据约束写作。论文先去重（DOI/arXiv ID 精确匹配，标题+作者+年份模糊匹配），再按方法和任务聚类；分析产物按论文 ID 合并，缺失字段保留为空而不是推测。对比产物将相似性、差异、时间线和矛盾分别记录；当不同论文对同一问题给出冲突结论时，系统保留双方观点、实验条件与证据，不强行投票生成单一结论。

为支持可解释融合，系统为证据计算可配置的质量分：$Q(e) = 0.30\times Relevance(e) + 0.25\times Credibility(e) + 0.25\times Verifiability(e) + 0.20\times Recency(e)$。其中 Relevance 来自检索/重排序分数，Credibility 考虑来源类型和同行评审状态，Verifiability 考虑是否存在原文定位，Recency 按时间衰减。权重在开发阶段仅作为初始值，最终通过验证集校准。声明 c 的证据支持度定义为 $Support(c) = \Sigma\ w_e\times Entail(e,c) / \Sigma\ w_e$；初始设计以 0.75 为自动通过阈值，0.50—0.75 进入人工复核，低于 0.50 则拒绝写入正文。

表 9 结果融合使用的核心产物契约

产物	生产者	关键字段	消费者
PlanArtifact	协调智能体	subqueries、filters、budget、task_dag	检索/编排器
RetrievalArtifact	检索智能体	papers、evidence、coverage、source_stats	分析/对比
AnalysisArtifact	分析智能体	method、dataset、metrics、innovation、limitation	对比/撰写
ComparisonArtifact	对比智能体	clusters、similarities、differences、trend、conflicts	撰写/校验
DraftArtifact	撰写智能体	outline、sections、claims、citation_refs	校验/前端
VerificationArtifact	引用校验智能体	claim_support、unsupported_claims、locator_errors	协调/撰写

合并策略强调“可追溯的保守写作”。如果两篇论文的实验条件不同，系统不会直接给出“方法 A 优于方法 B”的结论，而是写成“在数据集 D、指标 M 和设置 S 下，A 报告了较高结果；B 的实验条件为……”。对于只有摘要证据的论文，系统降低其可信度并标注“仅摘要证据”。这类保守表达会牺牲部分文风流畅度，但能减少把不同实验口径混为一谈的风险。

3.3.6 面向科研文本的 RAG 策略

本项目的 RAG 不是简单的“向量检索 + 拼接提示词”，而是由数据接入、结构化切分、混合召回、重排序、证据绑定和引用校验组成的流水线。系统优先使用开放获取的全文；无法获得全文时，仅使用摘要、标题和公开元数据，并在证据对象中记录 evidence_scope。论文按摘要、引言、方法、实验、结果、局限和结论等章节切分，保留页码、段落和表格标题等定位信息。该设计使引用不仅能看到“出自哪篇论文”，还能定位到“哪一节、哪一段、哪张表”。

表 10 科研文本 RAG 流水线设计

阶段	设计要点	关键参数/策略	输出
数据接入	arXiv/PubMed/Crossref 适配器，记录来源、许可和抓取时间	分页、退避、缓存；开放全文优先	标准化 Paper 元数据
清洗去重	DOI/arXiv ID 精确匹配；标题+作者+年份模糊匹配	版本合并；预印本与正式版标注	去重后的论文集合
章节切分	保留章节、页码、段落和表格标题	500—900 tokens；100—150 overlap	DocumentChunk + locator
索引构建	BM25 倒排索引 + 稠密向量索引	bge-m3 或可替换嵌入模型；pgvector	可检索证据库
混合召回	查询扩展后并行执行稀疏/稠密检索	元数据过滤：年份、来源、类型	Top-k 证据候选
重排序	交叉编码器或 LLM 重排，兼顾相关性与多样性	召回 50 → 重排 12；按论文去冗余	证据上下文
生成与校验	写作时绑定 evidenceId；校验 entailment 和 locator	支持度阈值；失败定向重试	可追溯报告
增量更新	按主题保存语料快照；支持重新索引	定时/手动刷新；版本哈希	可复现语料版本

混合召回阶段，系统对每个子查询分别取 BM25 Top-k 和稠密向量 Top-k，再以加权分数合并：$Score(q,e) = \lambda \times BM25(q,e) + (1-\lambda) \times \cos(Emb(q), Emb(e))$，$\lambda$ 在验证集上调。候选证据经过重排序后，还要执行论文级和章节级去冗余，避免上下文被同一篇论文的重复片段占满。上下文组

装采用“证据卡片”格式，每张卡片包含 evidenceId、论文标题、年份、章节、原文片段和定位；写作智能体只能引用卡片中的 evidenceId。若论文较长且证据分散，系统采用多轮检索而不是一次性塞入全部全文[11]。

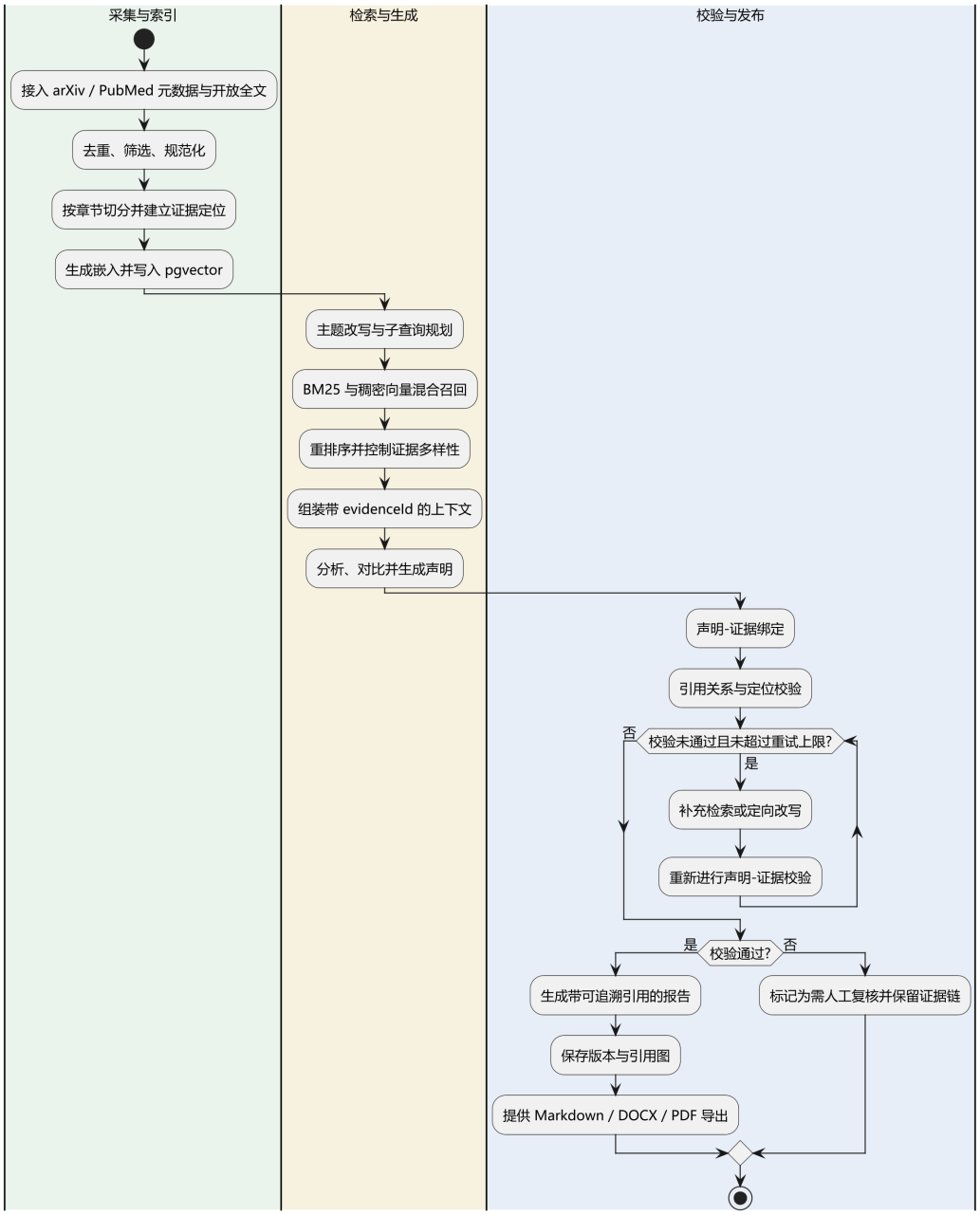


图 3 RAG 检索、生成与引用校验活动图

引用校验阶段同时检查三类问题：引用对象是否存在、引用内容是否支持声明、引用定位是否可回到原文。校验器可以组合 NLI/ entailment 模型与 LLM 判断，但必须保存模型版本和判定理由。对于“方法比较”“因果结论”“最新进展”等高风险声明，系统要求至少两条独立证据或明确限定条件；对于常识性过渡句，可以标记为“无需引用”，但不能把事实性声明混入其中。

3.3.7 领域模型与数据设计

图 4 给出项目的核心领域类图。综述项目聚合检索计划、草稿、任务和评测报告；检索计划与论文是一对多关系；任务运行记录用于恢复和审计。图 5 进一步给出证据—声明—引用模型：论文由多个证据片段组成，证据支持声明，声明绑定引用，引用再指回证据。通过这种双图结构，系统既能表达“哪些论文被纳入综述”，也能表达“每条结论由哪段原文支撑”。

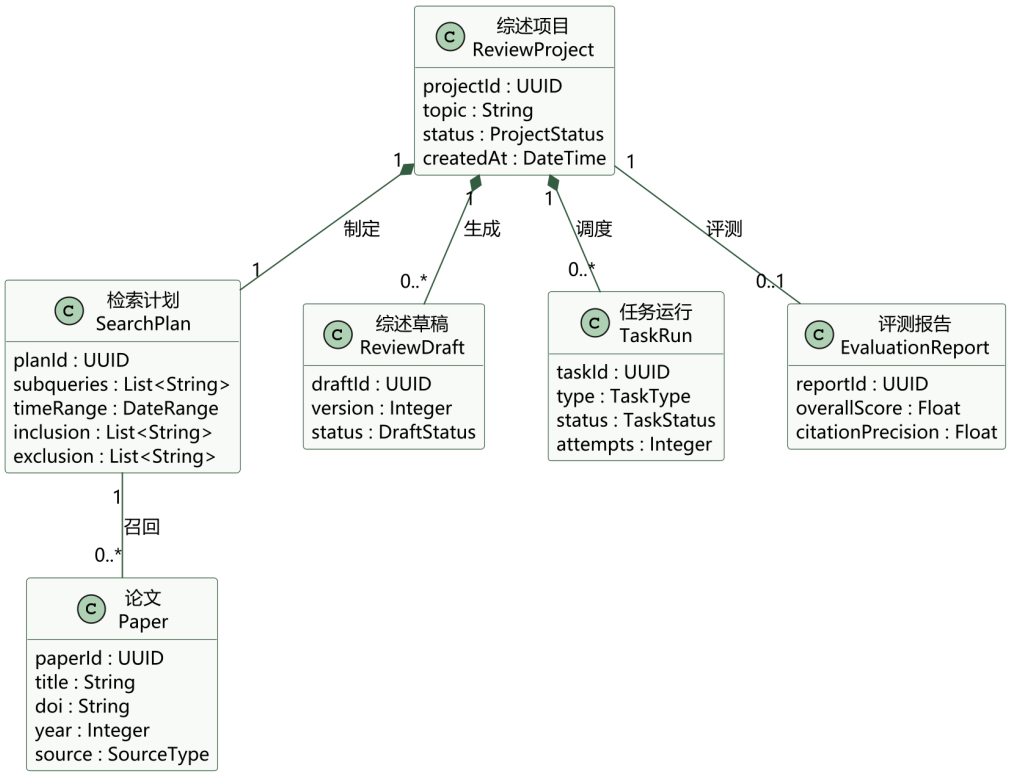


图 4 综述项目核心领域类图

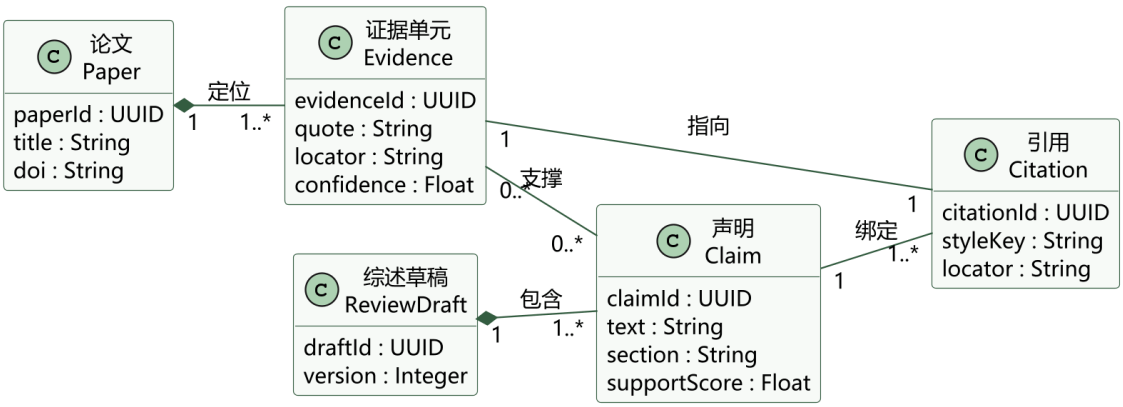


图 5 证据、声明与引用类图

数据持久化采用 PostgreSQL + pgvector 作为主存储：项目、论文、证据、声明、引用、任务和评测使用关系表保存；证据向量存储在 pgvector 扩展中；PDF、导出的报告和大型中间文件放入

对象存储；Redis 用于任务队列、短期缓存和进度事件。所有表和主要字段都带项目隔离标识，删除项目时应级联清理关联证据和文件。

表 11 主要数据对象与存储设计

对象/表	关键字段	用途	存储策略
review_projects	project_id、topic、status、config、owner_id	项目与配置	PostgreSQL，按 owner 隔离
papers	paper_id、doi、source、title、year、peer_reviewed	论文元数据	PostgreSQL；唯一约束去重
document_chunks	chunk_id、paper_id、section、locator、text、embedding	切分证据	PostgreSQL + pgvector
evidence	evidence_id、chunk_id、quote、confidence、scope	可引用证据	PostgreSQL
claims / citations	claim_id、text、support_score、evidence_id、style_key	声明与引用关系	PostgreSQL，外键约束
task_runs	task_id、type、status、attempts、checkpoint	任务状态与恢复	PostgreSQL + Redis 队列
artifacts	artifact_id、type、version、payload_ref	智能体结构化产物	PostgreSQL 元数据 + 对象存储负载
evaluations	metric、score、rater_id、notes	自动/人工评测	PostgreSQL，保留匿名评分

3.3.8 主业务流程与顺序设计

图 6 给出从主题输入到报告展示的主要顺序。系统在创建任务后先生成检索计划，再按子任务循环检索、分析和对比，随后生成初稿并进行引用校验；若存在不支持声明，则进入定向改写分支，直到通过或达到重试上限。整个过程的阶段事件通过 SSE 推送到前端，用户无需轮询。

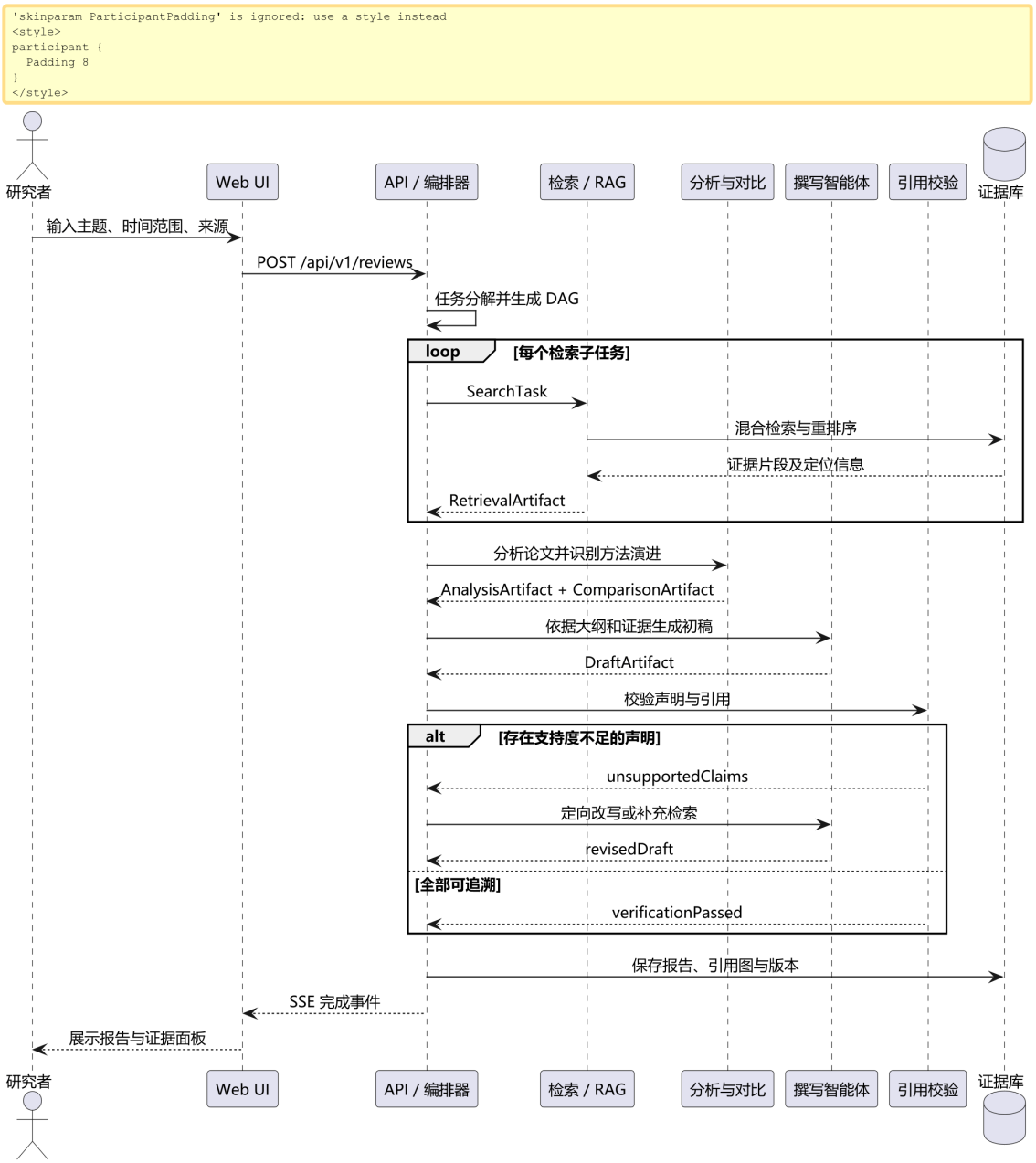


图 6 综述生成主流程顺序图

3.3.9 Web 界面与接口设计

前端采用 React + TypeScript，按“输入—过程—报告—证据”组织为四个核心视图。主题输入页提供数据源、年份、篇幅、语言和引用格式等配置；任务页显示阶段、进度、日志和预计成本；报告页支持章节导航、正文编辑、引用编号和导出；证据面板按 claimId/evidenceId 展示原文、论文元数据和定位。移动端只保证只读查看和基本操作，复杂编辑以桌面端为主。

表 12 Web 界面信息架构

页面/视图	核心内容	主要交互	对应需求
主题输入	自然语言主题、数据源、时间范围、篇幅、语言、引用格式	保存模板、校验输入、提交任务	FR-01、FR-02
任务进度	阶段时间线、当前任务、失败原因、token/成本	取消、重试、查看日志	FR-12
报告工作区	章节树、正文、引用编号、版本对比	编辑、保存版本、导出	FR-08、FR-10、FR-11
证据面板	原文片段、论文元数据、章节/页码、支持度	按论文/章节筛选、跳转报告声明	FR-09、FR-10
评测面板（可选）	自动指标、人工评分、消融结果	导出评测数据	NFR-04、NFR-06

表 13 核心 REST/SSE 接口

方法	路径	用途	主要返回
POST	/api/v1/reviews	创建综述任务	job_id、初始状态
GET	/api/v1/reviews/{id}	查询任务状态与阶段	status、progress、error
GET	/api/v1/reviews/{id}/events	SSE 进度事件流	stage、message、percent
GET	/api/v1/reviews/{id}/report	获取报告与引用图	sections、citations、version
PATCH	/api/v1/reviews/{id}/report	保存人工编辑	new_version
GET	/api/v1/evidence/{id}	读取证据原文和定位	quote、paper、locator
POST	/api/v1/reviews/{id}/export	导出 Markdown/DOCX/PDF	download_url
POST	/api/v1/reviews/{id}/cancel	取消任务	cancelled/current_state

3.3.10 部署与运行设计

原型采用 Docker Compose 进行本地/课程服务器部署。前端构建为静态资源，由 Nginx 提供服务；后端使用 FastAPI + Uvicorn；Celery Worker 执行长时间生成任务；PostgreSQL + pgvector 保存业务数据与向量；Redis 提供队列和事件缓存；MinIO 保存 PDF、报告和中间产物。LLM、arXiv、PubMed 通过可配置适配器访问，密钥只存在于后端环境变量或密钥管理服务中。

表 14 原型部署组件

组件	推荐技术	职责	部署方式
前端	React 18 + TypeScript + Vite	主题配置、进度、报告和证据面板	Nginx 静态站点
API/编排	FastAPI + Pydantic v2 + CrewAI	REST、任务编排、智能体调用	Uvicorn 容器
异步任务	Celery + Redis	长任务队列、重试和事件	Worker 容器
业务数据	PostgreSQL 16 + pgvector	关系数据、全文检索、向量索引	数据库容器/托管服务
对象存储	MinIO	PDF、报告、产物负载	对象存储容器
外部服务	可替换 LLM、arXiv、PubMed、Crossref	模型推理与文献元数据	HTTPS API

3.3.11 异常处理、可观测性与安全

异常处理遵循“记录—分类—重试—降级—人工介入”的顺序。数据源超时或限流时，系统按指数退避重试，并在覆盖率不足时提示更换数据源；检索结果过少时，扩大时间范围或使用同义词；智能体输出不符合 schema 时，先进行一次修复提示，仍失败则标记任务失败而不把无效文本传给下游；引用校验失败时，返回 unsupportedClaims 并在报告中保留问题标记。

可观测性方面，每个任务记录 trace_id、阶段耗时、模型/token 使用量、检索命中数、失败原因和重试次数；前端展示用户可理解的进度，研发侧可通过结构化日志定位问题。安全方面，落实 API 鉴权、项目级 RBAC、输入长度和速率限制、日志脱敏、密钥隔离和导出权限检查；对上传文献和个人数据按最小必要原则保存，并提供项目删除功能。

3.4 拟采用的开发方法、环境与测试方案

3.4.1 开发方法与技术环境

项目采用迭代式敏捷开发，以两周为一个迭代，每个迭代都产出可运行增量。开发过程遵循需求—设计—实现—测试—评审的闭环：需求变更写入 backlog；数据模型和接口契约先于实现确定；智能体提示词与输出 schema 纳入版本控制；每次合并前运行单元测试和静态检查。由于 LLM 输出具有不确定性，团队将“可重复的确定性测试”和“带容差的评测”分开管理，避免把偶发成功当作功能完成。

表 15 开发环境与技术栈

层面	技术选型	说明
开发语言	Python 3.11+、TypeScript	后端/智能体使用 Python，前端使用 TypeScript
多智能体框架	CrewAI	角色、任务和 Crew 编排；结构化输出配合 Pydantic
后端框架	FastAPI、Pydantic v2、SQLAlchemy	REST、SSE、数据校验与 ORM
前端	React 18、Vite、TypeScript	主题配置、报告编辑和证据面板
数据与检索	PostgreSQL 16、pgvector、BM25、bge-m3/可替换嵌入模型	关系数据、向量索引与混合检索
异步与存储	Redis、Celery、MinIO	任务队列、缓存、PDF/报告对象存储
测试与部署	pytest、Playwright、Docker Compose、GitHub Actions	单元/集成/E2E 与持续集成
LLM 接入	OpenAI-compatible API / 本地模型适配器	通过 LLM 网关解耦供应商

3.4.2 测试方案

测试分为四个层次：单元测试验证纯函数和关键规则，例如去重、过滤、证据评分、引用格式和 schema 校验；契约测试验证智能体输出、外部数据源适配器和 API 响应是否符合约定；集成测试验证检索、数据库、队列和导出链路；端到端测试使用 Playwright 覆盖“输入主题—查看进度—打开报告—点击引用—导出”的主路径。对智能体行为采用固定语料和 mock LLM 的回归测试，对真实模型输出采用离线评测集和人工抽检。

表 16 测试层级与通过标准

测试层级	覆盖对象	方法/工具	通过标准
单元测试	去重、筛选、评分、引用格式、权限	pytest + fixture	核心模块覆盖率 ≥ 70%；关键规则全部通过
契约测试	Agent 输出、REST、数据源适配器	Pydantic schema、mock server	字段完整、类型正确、错误可定位
集成测试	检索→分析→写作→校验→持久化	Docker Compose + 测试数据库	主链路可重复执行；失败可恢复
端到端测试	Web 主流程和导出	Playwright	核心路径无阻断错误；导出文件可打开
质量评测	生成内容与引用	自动指标 + 专家盲评	达到预设的引用/覆盖/满意度阈值

3.4.3 三领域评测设计

为满足“在 2—3 个计算机领域子方向上评估综述质量”的要求，项目选择检索增强生成、LLM 多智能体系统和软件漏洞检测三个子方向。三者分别代表“方法/系统型主题”“协作机制型主题”和“应用安全型主题”，能够检验系统在不同论文结构、评价指标和术语体系下的适应能力。每个方向选取 3 个代表性主题，每个主题准备 1 份专家关键点清单和 1 组引用核验样本；候选论文由系统检索，人工只负责最终核验，不提前把答案提供给系统。

表 17 三个计算机子方向的评测设置

子方向	示例主题	评测重点	基线/消融
检索增强生成	RAG 的忠实性评估与混合检索	检索覆盖、证据绑定、引用精度	Direct LLM；Single-agent + RAG
LLM 多智能体系统	多智能体任务分解与协作机制	方法分类、协作机制对比、演进脉络	Multi-agent without RAG；Ours
软件漏洞检测	基于深度学习的漏洞检测与数据集偏差	实验条件对比、数据集/指标抽取、局限识别	Single-agent + RAG；Ours without verifier

评测指标分为五组。检索组：Recall@20、nDCG@10、来源覆盖率和去重准确率；引用组：引用精度（被支持引用数/引用总数）、引用召回（有证据声明数/应引用声明数）、定位准确率和关键声明证据绑定率；内容组：关键方法覆盖率、方法分类一致性、演进脉络完整性和局限识别率；写作组：结构完整性、连贯性、批判性和可读性；效率组：端到端时延、token 成本和任务成功率。引用精度和证据绑定率作为一等指标，不能仅用文本相似度代替。

自动评测采用 RAGAS/ARES 等框架提供的忠实性、答案相关性和上下文指标作为辅助[12][13]，同时开发面向综述的规则检查：每条关键声明是否存在 evidenceId、引用是否能定位、数字/数据集/指标是否与原文一致。人工评测由 3 名具备相关背景的评分者独立完成，使用 5 分制量表（1=明显错误，5=可直接作为研究材料），随机盲评系统输出与基线输出。评分前进行样例校准，报告 Krippendorff's α 以检查一致性；目标 α ≥0.70，若低于阈值则修订评分量表并重新抽检[22]。对主要指标采用配对比较或 Wilcoxon 符号秩检验，报告效应量和置信区间。

3.4.4 验收标准

表 18 项目验收标准

验收项	最低标准	验证方式
端到端功能	主题输入到报告导出主流程可用	Playwright E2E + 演示脚本
多智能体协作	至少 4 类专业智能体 + 协调/校验角色；任务可恢复	代码审查 + 失败注入测试
RAG	混合检索与重排序可配置；引用可回溯到证据	检索评测 + 引用抽查
任务融合	结构化产物 schema 完整；冲突观点可保留	产物校验 + 案例审查
Web 原型	支持主题配置、进度、证据查看、编辑和导出	功能验收清单
三领域评测	完成 3 个方向、每方向至少 3 个主题	评测报告与原始记录
引用质量	关键声明 evidence 绑定率 100%；引用精度目标 ≥ 85%	人工核验 + 自动检查
可用性与性能	主流程无阻断错误；单次任务达到目标时延/预算	压力测试与成本统计

3.5 技术难度及特色分析

3.5.1 主要技术难度与应对措施

表 19 技术难点与应对措施

难点	具体表现	应对措施
学术数据异构	来源字段不一致、分页/限流、全文缺失	适配器 + 统一 schema + 缓存 + 明确 evidence_scope
论文结构解析	多栏排版、表格/公式、章节边界	结构化解析器；保留页码/章节；必要时只使用摘要
检索与上下文	长文本信息丢失、证据分散、上下文预算有限	混合检索 + 重排序 + 多轮检索 + 证据卡片[10][11]
多智能体误差累积	职责漂移、重复检索、错误传递	DAG、schema、共享黑板、置信度阈值和失败隔离[4][5]
引用可验证性	引用存在但不支持声明，或定位错误	claim-evidence 校验、原文定位、人工复核入口
评测主观性	不同评分者对覆盖度和连贯性判断不一致	量表、盲评、多人评分和一致性统计[22]
成本与稳定性	长流程 API 调用多、重试放大成本	模型分级、缓存、预算控制、断点续跑

3.5.2 项目特色与创新点

（1）角色—任务—产物三层编排。项目不只设置多个角色，还为每个任务定义契约、依赖和验收条件，用任务图取代无界对话，适合长流程科研写作。

（2）证据优先的混合 RAG。系统将元数据过滤、BM25、稠密检索、重排序和章节定位组合起来，并把 claim-evidence 绑定作为写作约束，而不是事后补引用。

（3）可解释的结果融合。分析、对比和写作产物统一进入共享黑板；冲突观点保留实验条件和来源，不追求表面一致的“唯一答案”。

（4）跨三领域、可重复的评测。除引用与覆盖指标外，同时测量时延和成本，使用人工盲评与评分者一致性检验，避免只展示“看起来不错”的样例。

（5）完整的 Web 工程闭环。系统覆盖输入配置、异步任务、进度推送、证据查看、报告编辑、版本管理和多格式导出，可作为后续科研知识管理工具的工程底座。

上述设计把多智能体协作、混合 RAG、领域数据建模和 Web 工程连接为一个闭环。后续实现将优先保证证据链完整和任务可恢复，再优化写作风格与界面细节；三领域评测结果将用于校准任务分解粒度、检索参数和融合阈值。

四、工作进度的大致安排

4.1 进度计划

项目以开题通过为 T0，按 24 周安排。进度表采用“阶段—时间—主要工作—成果形式”组织，阶段成果均可检查；若某一阶段延期，优先压缩非关键展示功能，不降低引用校验和评测的可复现要求。

表 20 项目进度安排

阶段	时间	主要工作	阶段成果
P1 调研与需求冻结	第 1—2 周	文献调研、用户访谈、需求优先级和验收标准	需求规格说明书 v1、文献矩阵
P2 架构与设计	第 3—4 周	总体架构、数据模型、接口契约、UML 图、评测方案	概要设计文档、UML 源文件
P3 数据与 RAG	第 5—8 周	arXiv/PubMed 适配、解析切分、向量索引、混合检索与重排序	可运行索引流水线、检索评测报告
P4 多智能体编排	第 9—12 周	CrewAI 角色、任务 DAG、共享黑板、结果融合、引用校验	智能体工作流、产物 schema、单元测试
P5 Web 原型	第 13—15 周	FastAPI、异步任务、SSE、前端页面、报告编辑与导出	Alpha 版 Web 应用、接口文档
P6 三领域评测	第 16—18 周	构建主题/关键点/引用样本，运行基线与消融，专家盲评	评测报告 v1、原始数据与脚本
P7 优化与稳定性	第 19—20 周	性能、成本、错误恢复、权限和可用性改进	Beta 版、缺陷清单与修复记录
P8 结项与答辩	第 21—24 周	技术报告、演示视频、最终验收、答辩材料	最终系统、技术报告、演示与答辩 PPT

4.2 预期成果及特色

项目预期形成一套可运行、可评测、可复现的科研文献智能综述原型。与只提供聊天式写作的工具相比，本项目的核心特色是证据链完整、任务过程可观测、结果融合有约束，并且能够在三个不同计算机子方向上给出量化对比。

表 21 预期成果清单

成果类别	具体内容	验收/展示方式
软件系统	CrewAI 多智能体编排、混合 RAG、Web 前端、REST/SSE 接口、Docker Compose	本地或课程服务器部署；现场演示
代码与文档	后端/前端源码、数据库 schema、提示词版本、API 文档、部署说明	Git 仓库与 README
评测材料	三个子方向的主题、关键点清单、引用核验样本、自动指标和人工评分脚本	评测报告与可复现实验脚本
技术报告	问题背景、调研、需求分析、UML 设计、实现、评测、局限和展望	不少于 15,000 字的中文报告
演示材料	端到端演示视频、答辩 PPT、典型成功与失败案例	答辩现场展示

表 22 预期质量指标

质量维度	目标值（初版）	说明
关键声明证据绑定率	100%	所有事实性关键声明必须有 evidenceId；无证据内容不得进入正式正文
引用精度	≥ 85%	随机抽取引用，由人工判断证据是否支持声明
关键点覆盖率	≥ 80%	相对专家关键点清单计算
专家综合评分	≥ 4.0/5	3 名评分者盲评的结构、覆盖、引用和批判性综合分
任务成功率	≥ 95%	在 30 次端到端运行中统计；失败可恢复也算成功
端到端时延	≤ 10 min	40—60 篇候选论文、正常外部 API 条件下的目标值
单次 token 成本	按预算配置	记录并报告；超预算暂停而非静默继续

上述数值是项目初期的工程目标，不是未经验证的承诺。若真实实验中无法达到，将报告实际结果、失败原因和改进口径，而不是选择性展示样例。项目的最终价值在于形成一套可复用的工程方法和诚实的评测证据，为后续扩展更多学科、模型和数据源提供基础。

导师意见

（对选题和工作过程及成果进行说明，并给出成绩。）

导师签名：_____

年　　月　　日

答辩小组意见

答辩组长签名：_____

年　　月　　日